Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



## Team Details

a. Team name: gTaprobane
b. Team leader name: Yogendran Nadaraja
c. Problem Statement:  A bank's headline discount rarely applies to the whole basket. The real saving is the basket total after each offer's exclusions, minimum bill, and cap, and no shopper can work that out at the till.

## Brief about the idea

- Savvy Cart AI is a grocery basket optimizer for Sri Lankan households that understands offer conditions. Gemini reads a household's receipts; the pantry works out what is running low from the gaps between purchases
- Deterministic code then plans the cheapest valid way to buy it on a chosen day: across up to four stores, one card per bill, with every exclusion, minimum bill, cap, and store or card rule applied exactly. Nothing is guessed by a model
- Offer Reality Check puts a bank offer's headline next to what its terms actually pay on the shopper's own basket
- It is live on Cloud Run with Gemini and the Agent Development Kit (ADK): a five-step guest flow with no login, ending in a MOCK approval. No order is placed

## Opportunities

- **How it's different**: Price-comparison apps find the cheapest store for each item. Savvy Cart prices the whole basket after each card offer's exclusions, minimum bill, cap, and no-combining rule. It also shows, line by line, when a "10% off every Saturday" pays LKR 0
- **How it solves the problem**: Gemini turns receipts and offer text into structured data. An exact OR-Tools optimizer assigns items to stores and applies every rule. The shopper compares the plan with shelf price, sees the best day to shop, and approves. Nothing is ordered automatically
- **USP**: AI reads, code counts. Every rupee comes from deterministic code. Two grounding layers remove any price, discount, or total the engine did not produce, so the plan can be trusted

## List of features offered by the solution

- Receipt-driven pantry: due dates come from the median gap between real purchases, with no forecasting model
- Receipt reading with Gemini: a photo becomes catalog-matched lines, each with a confidence score
- Offer Reality Check: Gemini reads a bank offer's text or screenshot into validated terms, which the shopper confirms
- Exact multi-store optimizer (OR-Tools CP-SAT): up to four stores, store promotions, one card offer per bill, and the cost of an extra trip
- Ask the agent (ADK): questions in plain language, re-planning on request, and a visible trace of every tool call
- Honest by design: it says "this offer pays LKR 0" when it does, never invents a number, approval is MOCK, and it never handles card numbers

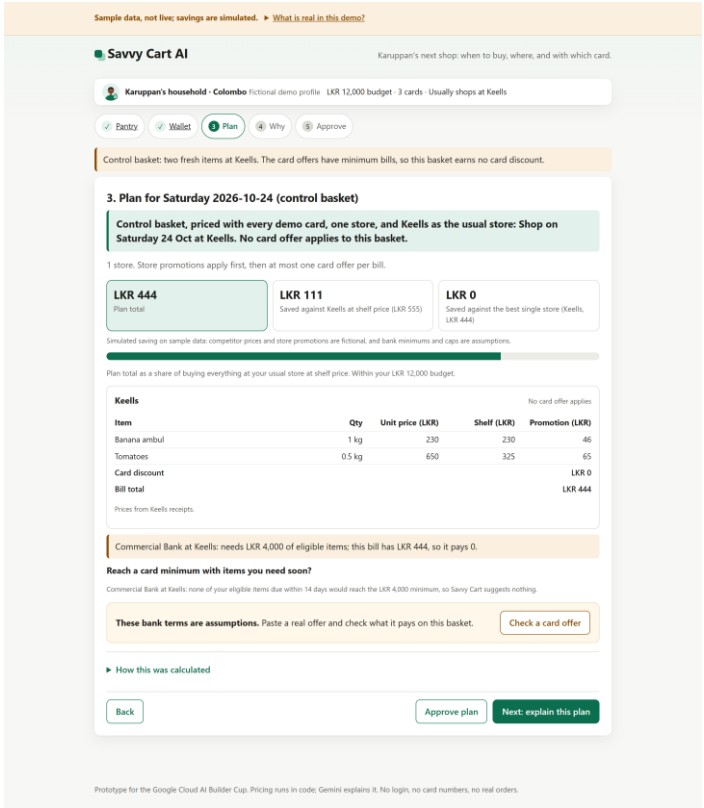
## Process flow diagram or Use-case diagram




A guest opens the live URL, with no login:

- Pantry: purchase history shows what is running low. Add a receipt, either the sample or a photo read by Gemini
- Wallet: pick cards, stores, and the cost of an extra trip, then test a bank offer with Offer Reality Check
- Plan: OR-Tools picks the stores, promotions, best card offer, and best day
- Why: the ADK agent explains the plan, re-plans on request, and shows every tool it called
- Approve: a MOCK order preview. No order is sent

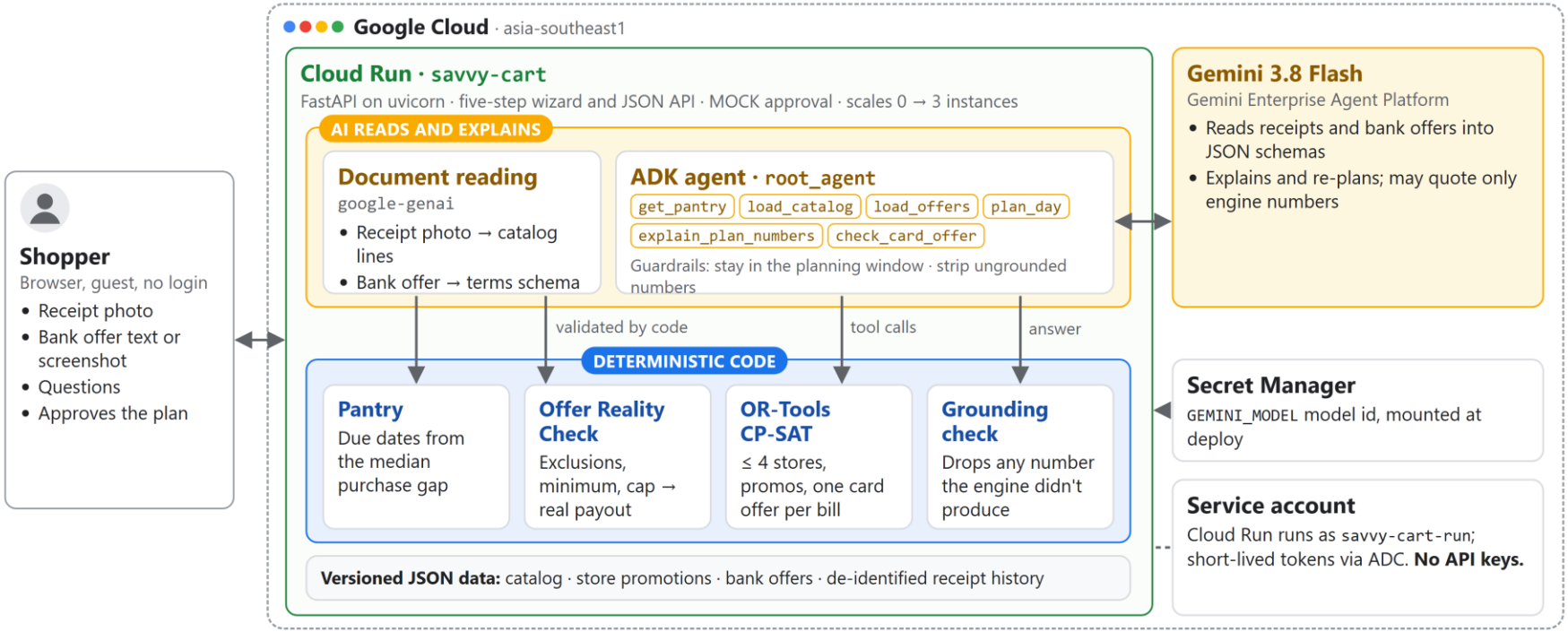
# Wireframes/Mock diagrams of the proposed solution




## Offer Reality Check: when a card offer pays nothing

- A small Saturday basket at Keells: banana ambul and tomatoes, LKR 444 after store promotions
- The Commercial Bank offer needs LKR 4,000 of eligible items. The bill has LKR 444, so the card discount is LKR 0, shown in amber
- Savvy Cart then checks whether anything the household needs in the next 14 days would reach the minimum. Nothing would, so it suggests nothing instead of pushing extra spending
- The bank's minimum is a labeled assumption. The shopper can paste the real offer and check it against this basket

# Architecture diagram of the proposed solution




**Gemini reads documents and explains.** Deterministic code owns every price, discount, and total.

# Technologies to be used in the solution

- Google Cloud: Cloud Run (asia-southeast1, scales to zero), Gemini gemini-3.8-flash on the Gemini Enterprise Agent Platform, Secret Manager, and service-account authentication with no API keys
- GenAI and agents: Agent Development Kit (google-adk 2.x) and google-genai
- Optimization: Google OR-Tools CP-SAT, an exact solver
- Application: Python 3.12, FastAPI, uvicorn, server-rendered HTML with a small vanilla JS/CSS layer, and httpx
- Build and test: Docker (python:3.12-slim, non-root) and pytest, with 120 tests passing

# Estimated implementation cost

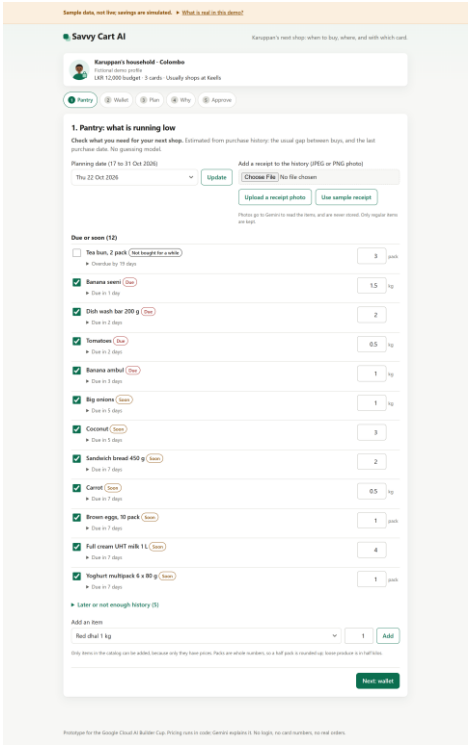
Prototype (today):

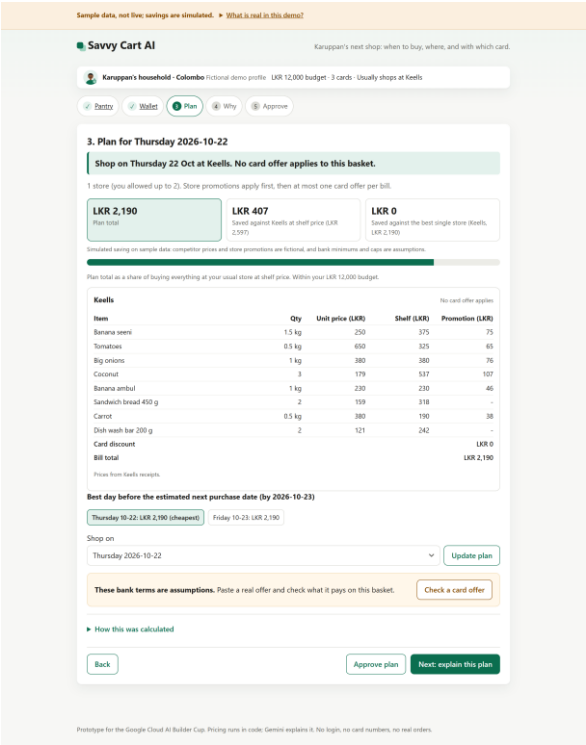
- Cloud Run scales to zero, with at most 3 instances, so there are no idle instances to pay for
- The only Gemini Flash traffic is from judges
- There is no database cost, because the data is committed fixtures
- Estimated spend over the evaluation window: single-digit to low tens of USD

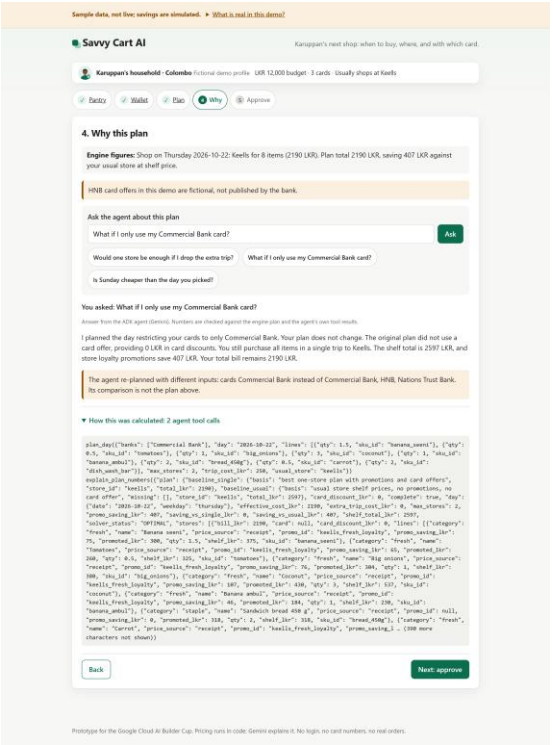
Production pilot (cost drivers):

- Gemini calls per household per week (receipts, offers, questions)
- Cloud Run request time
- Firestore reads and writes
- Licensed retailer and issuer data feeds.

# Snapshots of the prototype




**Pantry**: what is running low, from receipt history.

**Plan**: every bill line by line, with the best day to shop.

**Why**: the agent's answer and its tool calls.

# Prototype Performance report/Benchmarking

- 120 automated tests pass, covering pricing, offer eligibility, and guardrails
- Live Gemini latency (18 Oct 2026): reading a bank offer takes about 5–7 s, and the agent's answer takes 15–50 s. Almost all of that is the agent's sequential Gemini calls (3 to 7 per question); the optimizer and tools return in milliseconds. The shopper doesn't wait on it: the Why step opens in under 2 s with the engine's own explanation, and the agent runs only when the shopper presses Ask
- Worked example (simulated): the planning day is Thursday 22 Oct. Adding the sample Keells receipt drops the due list from 12 items to 9. With tea bun left unchecked, 8 items go into the plan. The engine plans one Keells trip for LKR 2,190, which is LKR 407 below Keells shelf price (LKR 2,597); all of that comes from store promotions. Two stores were allowed, but Keells alone is the cheapest option. No card offer applies, and asking the agent, "What if I only use my Commercial Bank card?" confirms the plan doesn't change
- Offer Reality Check: Seylan Bank's published "10% off every Saturday" suggests LKR 593 off this basket but pays LKR 0. The bill is LKR 2,475 short of the LKR 8,000 minimum
- All savings are simulated on labeled sample data

# Additional Details/Future Development

Roadmap:

- A pilot with Colombo households
- Faster agent answers: stop the model re-sending the whole plan to explain_plan_numbers (it reads it from the session instead), lower its thinking budget, and limit tool rounds. Our traces suggest this roughly halves answer time.
- Authorized retailer and issuer data feeds: versioned prices and real card terms, stored in Firestore
- Adapting the rules to other JAPAC markets: currencies, rounding, and local card offers
- Business model: free for households. Later, issuers or retailers pay for referrals the shopper has consented to and that are disclosed. Shopper data is never sold
- Out of scope for the prototype: live prices, real orders and payments, and partner checkout.

Provide links to your:

- GitHub Public Repository: savvy-cart-github-repo
- Demo Video Link : savvy-cart-demo-video
- Final Product Link: savvy-cart-product-link

Google Cloud H2S

# AI Builder Cup

Build in JAPAC.
Build for the world.

# Thank you

